

CORE ALGORITHM SERIES

Python Binary Trees

Efficient Hierarchical Data Management

Volume 04: Dynamic Structures

Published by: CodeHive Engineering

Module Overview

01. The Binary Advantage	3
02. Python Implementation	4
03. Special Classifications	5
04. Advanced Tree Properties	6
05. Traversal Theory (BFS vs DFS)	7
06. Deep Dive: Pre-order Logic	8
07. Deep Dive: In-order Logic	9
08. Deep Dive: Post-order Logic	10

01. The Binary Advantage

A **Binary Tree** is a hierarchical data structure where each node is restricted to a maximum of two child nodes: the Left Child and the Right Child.

Why the Two-Child Limit?

This simple constraint unlocks massive performance benefits in computer science:

- **Simplified Logic:** Searching, insertion, and deletion algorithms become predictable and easy to recursive.
- **Optimized Search:** In a Binary Search Tree (BST), lookup times drop to $O(\log n)$.
- **Balance:** Maintaining structural stability (e.g., AVL trees) is easier when child count is limited.
- **Memory Efficiency:** Binary trees can be mapped directly into arrays with mathematical precision.

Key Note: Unlike linked lists (one-way) or general trees (infinite ways), the Binary Tree provides the perfect balance between complexity and efficiency.

02. Python Implementation

Implementing a binary tree in Python mirrors the logic of a doubly linked list, but instead of "Previous/Next," we use "Left/Right."

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Constructing the hierarchy
root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')

# Link nodes
root.left = nodeA
root.right = nodeB

# Internal deep access
print(root.right.data) # Output: B
```

Linking Logic

Each `TreeNode` object acts as a container for data and two "slots" for memory addresses of other nodes. High-level construction involves creating objects and assigning them to the `left` or `right` attributes of a parent.

03. Special Classifications

Not all binary trees are built equally. Their "shape" determines their speed.

1. Balanced Binary Tree

A tree where for every node, the height difference between the left and right subtrees is at most one. This prevents the tree from becoming a linked list and keeps searches fast.

2. Full Binary Tree

A strict structure where every node has either **exactly 0 or 2 child nodes**. No node is left with just a single descendant.

Visual Insight: In a Full Binary Tree, internal nodes always branch out twice, creating a symmetrical "V" shape at every intersection.

04. Advanced Tree Properties

3. Complete Binary Tree

All levels are completely filled except possibly the last level. The nodes in the last level are filled from **left to right**. This is the optimal structure for implementing Binary Heaps.

4. Perfect Binary Tree

The "Gold Standard" of trees. All internal nodes have two children, and all leaf nodes are at the **exact same level**. It is simultaneously Full, Balanced, and Complete.

Property Math:

In a Perfect Binary Tree of height h , the total number of nodes is exactly $2^{(h+1)} - 1$.

05. Traversal Theory

Because trees are non-linear, you can't just slide from index 0 to N. You must choose a strategy to visit nodes.

Breadth-First Search (BFS)

Visits nodes level by level. You explore all neighbors at the current depth before moving deeper. This is typically implemented using a **Queue**.

Depth-First Search (DFS)

Moves down a branch to its deepest leaf before backtracking. DFS is usually implemented using **Recursion**. The three variations are:

- **Pre-order:** Root → Left → Right
- **In-order:** Left → Root → Right
- **Post-order:** Left → Right → Root

06. Deep Dive: Pre-order Logic

In Pre-order traversal, a node is visited **before** its descendants. It's often used to create a "clone" of a tree or to visualize prefix expressions.

```
def preOrderTraversal(node):  
    if node is None:  
        return  
  
    # 1. Visit Root  
    print(node.data, end=" ", "  
  
    # 2. Recurse Left  
    preOrderTraversal(node.left)  
  
    # 3. Recurse Right  
    preOrderTraversal(node.right)
```

Behavior

The algorithm dives down the left side completely, printing as it goes. Once it hits a leaf, it pops back up to explore the nearest right-hand branch.

07. Deep Dive: In-order Logic

This is arguably the most important traversal for Binary Search Trees. It visits nodes **after** the left subtree but **before** the right.

```
def inOrderTraversal(node):  
    if node is None:  
        return  
  
    # 1. Recurse Left  
    inOrderTraversal(node.left)  
  
    # 2. Visit Root  
    print(node.data, end=" ", "  
  
    # 3. Recurse Right  
    inOrderTraversal(node.right)
```

The Magic of BST: In a Binary Search Tree, an In-order traversal will always print the data in **sorted ascending order**!

08. Deep Dive: Post-order Logic

In Post-order traversal, the root node is visited **last**. This is mandatory for operations where you need to process children before the parent, such as deleting a tree or calculating file sizes in a directory.

```
def postOrderTraversal(node):  
    if node is None:  
        return  
  
    # 1. Recurse Left  
    postOrderTraversal(node.left)  
  
    # 2. Recurse Right  
    postOrderTraversal(node.right)  
  
    # 3. Visit Root  
    print(node.data, end=" ", " ")
```